01 / 전체 지도

먼저 전체 흐름을 잡기

브라우저는 요청하고, 서버는 처리하고, DB는 기억한다.

이 자료는 함수 하나씩 외우기 전에, 화면에서 한 행동이 어떤 코드와 데이터를 거치는지 이해하기 위한 안내서다. 예시 사용자 ID와 대화 ID는 모두 가상 값이다.

사용자 행동	서버에서 일어나는 일	C · 내가 구현
회원가입	A가 입력 검증·비밀번호 해싱	create_user로 사용자 저장
로그인	A가 비밀번호 확인·쿠키 발급	get_user_by_username으로 조회
보호된 기능 요청	A가 토큰을 검증해 사용자 확인	get_user_by_id로 DB 사용자 조회
질문 전송	B가 이전 문맥을 구성해 AI 호출	recent_turns로 성공 대화 조회
AI 처리 결과 수신	B가 성공·실패 결과 저장 요청	create_chat_log로 DB 저장
지난 대화 보기	D가 화면 표시, C가 JSON API 제공	list_chat_logs와 기록 조회 API

C · 내가 구현
내 역할은 AI 답변을 만드는 것이 아니라, 사용자·대화를 저장하고 필요한 범위만 정확하게 조회하는 DB 계층을 만드는 것이다.

담당	범위
A · 임대균	인증·회원가입·로그인 / yun-lim
B · 손재현	AI 호출·대화 처리 흐름 / sonjehyun123-maker
C · 최건영 (나)	DB 연결·모델(ChatLog)·CRUD·기록 API / 00skgun
D · 임익화	앱 연결·화면·공통 규격·배포 / zxcv718

구분해서 기억하기
읽는 순서: 1~5쪽은 기본 구조, 6~10쪽은 데이터 이동, 11~12쪽은 내 기여와 평가 답변. 민트색 영역과 'C · 내가 구현' 표시는 내 작업이다. 다른 담당 표시는 협업 흐름을 이해하기 위한 것이다.

코드·자료 근거: 담당 기준 / 앱 진입점

02 / 실행 환경과 DB 기반

사용자 요청 전에 준비되는 것

이름	쉽게 말하면	이 프로젝트의 역할
Vercel	프로그램을 실행하는 곳	FastAPI 앱을 서버리스 함수로 실행
FastAPI	요청을 받아 처리하는 틀	주소별 Python 함수와 인증·검증 연결
PostgreSQL	DB 시스템	users와 chat_logs 데이터를 보관
Neon	PostgreSQL 제공 서비스	앱 밖에서 개발·운영 DB를 제공
SQLAlchemy	Python DB 작업 도구	객체·쿼리·트랜잭션을 다룸
ORM	객체와 DB 테이블의 연결	ChatLog 객체로 대화 행을 다루게 함
psycopg	PostgreSQL 드라이버	Python과 실제 PostgreSQL 사이 통신

01 C
database.py
DATABASE_URL을 확인하고 engine, SessionLocal, Base를 준비한다. 주소가 비어 있으면 즉시 오류를 낸다.

02 D
main.py
FastAPI app을 만들고 pages·auth·logs·chat 라우터, 공통 오류 처리와 요청 ID 미들웨어를 등록한다.

03 A + C
보호된 요청의 DB 세션
A의 get_db가 C의 SessionLocal로 Session을 만들고, 작업이 끝나면 finally에서 닫는다.

C · 내가 구현
engine은 연결 관리 기반, SessionLocal은 Session을 만드는 공장, Base는 ORM 모델의 공통 부모다. 내가 만든 것은 이 DB 기반이다. Session은 DB 작업을 관리하는 객체로 로그인 쿠키와 다르다. 요청별 Session 생성·종료 코드는 A가 작성했다.

구분해서 기억하기
engine 생성은 즉시 DB 접속·테이블 생성 성공을 뜻하지 않는다. 테이블 생성은 별도 python -m scripts.create_tables --confirm 작업이다. 이 스크립트는 내가 작성했고 실제 Neon 실행은 D가 담당했다. 이 자료를 보며 운영 DB에서 실행할 필요는 없다.

왜 외부 DB인가? Vercel 함수의 로컬 파일은 영구 저장소가 아니므로 앱 내부 SQLite 파일에 대화를 계속 보관하는 구성은 맞지 않는다. 같은 Neon 브랜치에 연결하고 데이터를 삭제하지 않았다면 앱 재배포 후에도 기존 기록을 조회할 수 있다. [V]

코드·자료 근거: DB 기반 / 세션 수명 / 초기화 도구 / V · Vercel 저장소 설명

03 / 계정이 생기고 인증되는 과정

회원가입은 저장, 로그인은 확인

회원가입: POST /api/auth/register

01 A + D	**입력 받기** 회원가입 화면이 username·password JSON을 보낸다. 공통 RegisterRequest 형식으로 입력을 검증한다.
02 A	**비밀번호 해싱** hash_password가 비밀번호를 해시로 바꾼다. 평문 비밀번호 자체를 users에 저장하지 않는다.
03 C	**create_user(db, username, password_hash)** 이미 해시된 값을 받아 User 객체를 만들고 add → flush → refresh → commit으로 저장한다.
04 A	**가입 완료 응답** UserResponse로 id·username만 돌려준다(201). 화면은 /login으로 이동한다. 가입만으로 로그인 쿠키를 발급하지 않는다.

로그인: POST /api/auth/login

01 C	**get_user_by_username** 입력된 아이디에 해당하는 사용자 객체를 조회한다. 없는 사용자면 None이다.
02 A	**검증·토큰 발급** verify_password로 비밀번호를 확인한다. 성공하면 JWT를 access_token이라는 HttpOnly 쿠키에 넣고 200 응답을 보낸다.
03 A + D	**/chat으로 이동** 로그인 화면의 JavaScript가 /chat으로 이동한다. 이후 같은 사이트 요청에 브라우저가 쿠키를 보낸다.

C · 내가 구현
내 구현: 사용자 저장과 사용자명 조회 함수. User 모델, 해싱, 비밀번호 검증, JWT·쿠키 발급 자체는 A의 구현이다.

구분해서 기억하기
로그아웃은 POST /api/auth/logout이다. 현재 사용자를 확인한 뒤 쿠키를 만료시키고 204를 반환한다. users나 chat_logs 행을 삭제하는 동작이 아니다.

코드·자료 근거: 인증 API / 내 사용자 CRUD / 화면 제출 처리

04 / CurrentUser와 사용자별 접근 제어

서버는 내가 누구인지 어떻게 아나

사용자 ID는 주소창에서 믿고 받는 값이 아니라, 검증한 로그인 정보로 정한다.

01 브라우저	**보호된 요청 전송** 예: GET /api/me/chats. 로그인 때 받은 access_token 쿠키를 함께 보낸다.
02 D	**요청 추적** main.py 미들웨어가 request_id를 만들고 요청 로그에 연결한다. 사용자 ID와 요청 ID는 서로 다르다.
03 A	**get_current_user** 쿠키를 읽고 decode_access_token으로 JWT 서명·만료 등을 검증해 사용자 ID를 얻는다.
04 C	**get_user_by_id(db, user_id)** DB에서 그 ID의 User가 실제로 존재하는지 조회한다.
05 A → C	**CurrentUser 전달** 사용자가 확인되면 라우터가 user.id를 사용한다. 기록 API는 이 값으로 list_chat_logs를 호출한다.

```
# 실제 코드의 핵심만 발췌
logs = crud.list_chat_logs(
    db, user.id, limit=limit, offset=offset
)

# crud.py 안의 조회 조건
.where(ChatLog.user_id == user_id)
```

C · 내가 구현
내가 보호하는 지점: 로그인 사용자 ID로만 호출되는 기록 API + CRUD의 user_id 필터. user_id 외래키는 사용자 존재 관계를 지키는 장치이지, 그 자체가 타인 조회를 막는 인증 기능은 아니다.

확인 질문 · A(id=7)가 ?user_id=8을 주소에 붙이면 B의 기록을 볼 수 있을까?

아니다. 이 라우터는 user_id를 조회 인자로 받지 않고 CurrentUser의 user.id를 쓴다. 추가 쿼리문자열은 이 함수에서 사용하지 않는다. 실제 조건은 여전히 ChatLog.user_id == 7이다.

구분해서 기억하기
유효한 인증이 없으면 /api/* 요청은 401 JSON이다. /chat·/logs 같은 화면 요청은 /login으로 302 이동한다. 이 차이는 D의 공통 오류 처리에서 정한다.

코드·자료 근거: 인증 의존성 / 내 기록 API / 공통 오류 처리

05 / 정상 응답 경로 - 가장 중요한 페이지

질문 한 번이 처리되는 정확한 순서

가상 상황: 사용자 ID 7이 '리스트가 뭐야?'를 전송한다. 아래는 인증과 입력 검증을 통과한 정상 경로다. 인증·검증이 실패하면 AI 호출 전 중단된다.

01 D	**chat.js → POST /api/chat** {"message":"리스트가 뭐야?"}를 보낸다. 화면에는 대기 중인 질문·답변 영역을 만들고 중복 전송을 막는다.
02 A + D + B	**현재 사용자·입력 확인 → chat 라우터** CurrentUser와 DbSession, ChatRequest를 사용한다. 라우터는 await handle_chat(db, user.id, payload.message)를 호출한다.
03 C	**recent_turns(db, 7, n=5)** 사용자 7의 최근 성공 대화를 조회해 과거 → 현재 순서의 ChatLog 목록을 돌려준다.
04 B	**문맥 구성 → AI 호출** 각 기록을 user/assistant 메시지로 바꾸고 현재 질문을 덧붙인다. ai_client가 system 지시를 앞에 붙여 외부 AI API를 호출한다.
05 B → C	**답변 수신 → create_chat_log** B가 질문·답변·요청 ID·소요 시간 등을 전달한다. C 함수가 status=success, error_code=None으로 저장·커밋한다.
06 B → D	**성공 응답 → 화면 표시** 저장된 id로 {answer, chat_id}를 반환한다(200). chat.js가 답변을 표시한다. HTTP 응답은 DB 저장 코드가 성공한 뒤에 생성된다.

C · 내가 구현
이 한 번의 요청에서 내 핵심 작업은 두 번 등장한다: ① AI 호출 전에 이전 대화 조회 ② AI 호출 뒤 결과 저장. 현재 사용자 확인 때도 내 사용자 조회 함수를 재사용한다.

구분해서 기억하기
브라우저가 Neon이나 AI API에 직접 접속하지 않는다. 서버가 DB·AI를 호출하며 연결 비밀값은 브라우저에 전달하지 않는다. 이 자료의 코드는 실제 서비스 호출이 아니라 읽기 전용으로 확인했다.

코드·자료 근거: 질문 라우터 / B의 처리 흐름 / 화면 전송

06 / 내가 만든 recent_turns

최근 대화 5개는 어떻게 고르나

C · 내가 구현
함수 계약: recent_turns(db, user_id, n=5) → list[ChatLog]. 동기 함수이므로 await 없이 호출하며, dict가 아니라 ORM 객체 목록을 반환한다. async 함수 안에서 호출해도 DB 작업 자체가 자동으로 비동기가 되지는 않는다.

처리 순서	이유
1. user_id로 필터	다른 사용자의 대화가 섞이지 않도록
2. status=success 필터	AI 문맥에는 성공한 질문·답변 쌍만 넣도록
3. created_at DESC, id DESC	최근 것부터 고르고 시각이 같아도 순서를 고정하도록
4. LIMIT n	필터를 통과한 기록 중 최근 n개만 선택하도록
5. 결과를 뒤집기	AI에는 과거 → 현재 순서로 전달하도록

손으로 따라가기: n=2인 가상 예시

대화 ID / 시각 순서	사용자	결과
101 / 가장 과거	7	success
102	7	error
103	8	success
104	7	success
105 / 가장 최근	7	success

사용자 7의 성공 기록 = [101, 104, 105]. 최근 2개를 선택하면 [105, 104], 반환할 때 뒤집으면 [104, 105]다. 위 예시에서는 시각 순서와 ID 순서가 일치한다고 가정했다.

구분해서 기억하기
대화 1개는 질문 1개 + 답변 1개의 묶음이다. 이전 성공 기록이 5개라면 과거 메시지 10개 + 현재 질문 1개 + system 지시 1개가 AI에 전달된다. DB의 대화 행 5개와 AI 메시지 5개는 다르다.

확인 질문 · 이 함수만 만들면 AI가 이전 대화를 기억하나?

아니다. C는 DB 기록을 반환한다. B의 _build_messages가 객체의 question·answer를 메시지로 바꾸고 AI 요청에 넣어야 문맥이 전달된다. 이전 기록이 없으면 빈 목록을 반환한다.

코드·자료 근거: recent_turns 구현 / 메시지 변환 / 필터·정렬 검증

07 / 내가 만든 저장 트랜잭션

답변을 받았다고 저장된 것은 아니다

AI 답변 생성과 DB 저장은 서로 다른 작업이다.

```
# create_chat_log의 핵심 흐름 발췌
try:
    db.add(chat_log)
    db.flush()
    db.refresh(chat_log)
    db.commit()
except SQLAlchemyError:
    db.rollback()
    raise
```

단계	실제로 하는 일
객체 생성	user_id, question, answer 등으로 ChatLog 객체를 만든다. 아직 저장 확정은 아니다.
add	Session에 저장할 객체를 등록한다.
flush	INSERT 등의 SQL을 DB에 보낸다. 아직 commit 전이라 저장 확정은 아니다.
refresh	DB 값을 다시 읽어 id·created_at 같은 생성값을 객체에 반영한다.
commit	현재 트랜잭션의 변경을 확정한다.
rollback + raise	DB 오류가 나면 현재 실패한 작업을 되돌리고 오류를 호출자에게 알린다.

C · 내가 구현
저장 순서와 실패 처리는 내가 구현했다. refresh를 commit 전에 두어, DB 값을 다시 읽는 과정까지 실패했을 때 아직 미확정인 저장을 rollback할 수 있게 했다.

구분해서 기억하기
rollback은 과거에 이미 commit한 모든 대화를 지우는 기능이 아니다. 또한 commit은 객체 하나만이 아니라 현재 트랜잭션 전체에 적용된다. 사용자 생성 함수도 같은 저장 순서를 사용한다.

성공 경로에서는 저장된 ChatLog를 B에게 반환하고, B가 chat_log.id를 ChatResponse의 chat_id에 넣는다. DB 저장 오류면 정상 답변 200으로 처리하지 않고 오류 경로로 간다.

코드·자료 근거: 저장 코드 / 실패 단계 테스트 / 실제 DB 재사용 검증

08 / 테이블·외래키·식별자

사용자 한 명과 대화 여러 개의 관계

users는 사용자 한 명을 한 행으로 보관한다. chat_logs는 질문 한 번의 성공·실패 결과를 한 행으로 보관한다. 사용자 1명에게 대화 여러 개가 연결되므로 1:N 관계다.

테이블 / 가상 행	주요 값	해석
users	id=7, username=건영	사용자 1명
chat_logs	id=101, user_id=7	건영의 첫 번째 대화
chat_logs	id=102, user_id=7	같은 사용자의 두 번째 대화
chat_logs	id=103, user_id=8	다른 사용자의 대화

C · 내가 구현
내가 만든 ChatLog의 user_id는 users.id를 참조하는 외래키(FK)다. User 모델 자체는 A가 만들었고, 나는 User를 조회·저장하는 CRUD 함수를 만들었다.

ChatLog의 11개 컬럼 묶음	저장 이유
id · user_id · request_id	대화 자체, 소유 사용자, 서버 요청 추적을 구분
question · answer	질문·답변 원문 보관
status · error_code	성공·실패와 실패 종류 기록
provider · model	어떤 AI 제공자·모델을 썼는지 기록
latency_ms · created_at	AI 처리 구간 소요 시간과 DB 기록 생성 시각

구분해서 기억하기
id는 대화 기본키(PK), user_id는 소유 사용자, request_id는 요청 추적용이다. request_id에는 인덱스가 있지만 UNIQUE 제약은 없다. chat_id는 API 이름에 맞춰 id를 반환하는 속성이며 12번째 DB 컬럼이 아니다.

확인 질문 · 신규 사용자 한 명이 질문 10번을 모두 정상 저장하면 users와 chat_logs는?

그 사용자에 대해 users 1행, chat_logs 10행이다. 대화 10행은 같은 user_id를 가진다. AI 오류여도 오류 기록 저장에 성공하면 한 행이지만, DB 저장 자체가 실패하면 그 요청의 행은 남는다고 보장할 수 없다.

코드·자료 근거: 두 ORM 모델 / ERD·제약·인덱스 / API 응답 형식

09 / 최신 화면 흐름 반영

지난 대화를 보여주는 세 가지 경로

요청	어디서 조회하나	화면 / 응답
GET /chat	D pages.py가 C list_chat_logs(limit=50) 호출	최근 50개를 골라 뒤집고 HTML에 미리 표시
위로 스크롤	D chat.js가 C GET /api/me/chats?limit=50&offset=50 등 호출	더 오래된 묶음을 받아 위쪽에 삽입
GET /logs	D pages.py가 C list_chat_logs 기본값으로 호출	최근 20개를 최신순 HTML로 표시

C · 내가 구현
세 경로의 데이터 조회 기반은 내가 만든 list_chat_logs다. JSON API인 GET /api/me/chats도 내가 구현했다. /chat의 화면 복원·스크롤 로딩·말풍선과 /logs 템플릿 연결은 D가 구현했다.

API로 읽는 경우의 순서

01	A	**CurrentUser 확인** 검증된 로그인 사용자만 통과한다. 예: user.id=7.
02	C	**list_my_chats → list_chat_logs** 사용자 7의 성공·실패 기록을 created_at DESC, id DESC로 정렬하고 limit·offset을 적용한다.
03	C + D	**ORM → 응답 모델 → JSON** C 라우터가 D의 ChatLogItem으로 변환한다. FastAPI가 JSON으로 응답하며 D 화면이 표시한다.

구분해서 기억하기
AI 문맥 5개, 채팅 첫 화면 50개, /logs 기본 20개는 서로 다른 용도다. 첫 /chat 화면은 기록 API를 호출해서 만드는 것이 아니라, 서버가 CRUD를 직접 호출해 HTML을 만드는 SSR 방식이다.

확인 질문 · limit=50, offset=50은 무슨 뜻인가?

필터·정렬된 목록에서 앞의 50행을 건너뛰고 최대 50행을 가져온다. '대화 ID가 50 이상'이라는 뜻이 아니다. 동시 저장·삭제가 있으면 offset 방식의 페이지 경계는 달라질 수 있다.

브라우저 새로고침은 DB 기록을 다시 읽는 것이지 AI 답변을 새로 생성하는 것이 아니다. 로그아웃해도 DB 기록은 남아 같은 사용자로 다시 로그인하면 조회할 수 있다.

코드·자료 근거: 내 기록 API / 화면의 직접 조회 / 추가 로딩

10 / 오류 분기와 rollback 사례

어디서 실패했는지 구분하기

실패 위치	처리 흐름	대화 행 / 응답
인증·질문 검증	라우터 본문 전에 중단	대화 저장 없이 401 / 422
이전 문맥 DB 조회	recent_turns 오류가 바깥으로 전파	AI 호출·대화 저장 없이 500
AI 호출의 AppError	B가 오류를 잡고 C 저장 함수로 status=error, answer=빈 문자열 기록 시도	저장 성공하면 오류 행 / AI 오류는 503
AI 성공 후 DB 저장	C가 SQLAlchemyError에 rollback 후 재전파	저장 보장 불가 / 공통 500
AI 실패 + 오류 기록 저장도 실패	B가 저장 오류를 로그에 남기고 원래 AI 오류를 재전파	오류 행 보장 불가 / 원래 AI 오류
기록 조회 DB 오류	C API가 공통 INTERNAL_ERROR로 변환	빈 성공 []가 아닌 500

내가 설명할 트러블슈팅: 실패 후 같은 Session 사용

중복 아이디나 잘못된 외래키 때문에 flush가 실패하면 Session을 바로 계속 사용할 수 없다. create_user·create_chat_log에서 rollback 후 원래 예외를 전달하도록 구현했다.

C · 내가 구현
검증: 테스트용 PostgreSQL에서 일부러 저장 제약 위반을 만든 뒤, 같은 Session으로 정상 데이터를 저장·조회할 수 있는지 검사했다. 운영 장애를 직접 겪었다고 표현하지 않고, 실패 상황을 재현·검증했다고 설명한다.

구분해서 기억하기
'성공·실패 모두 저장한다'는 처리된 AI 결과를 기록하려는 설계다. 모든 요청의 DB 저장을 보장하지는 않는다. 브라우저에서 네트워크 오류가 보여도 서버 저장이 실패했다고 단정할 수 없다. 서버 로그와 DB 대화 기록도 서로 다르다.

자료는 실제 코드의 분기를 설명한다. chat_service.py의 주석과 달리 예외는 raise로 바깥에 전달되며 main.py의 공통 핸들러가 HTTP 오류 응답을 만든다. '오류를 전혀 던지지 않는다'고 설명하면 틀린다.

코드·자료 근거: B의 예외 분기 / 공통 오류 응답 / 사용자 rollback 검증 / 대화 rollback 검증

11 / 기능과 코드·이슈를 연결

내가 만든 부분만 다시 모아보기

내 작업	실행 과정에서 쓰이는 곳	근거
engine · SessionLocal · Base	DB 작업의 연결·세션·모델 기반	#70 / PR #77
get_user_by_username · get_user_by_id · create_user	가입, 로그인, 보호된 요청의 사용자 확인	#66 / PR #85
ChatLog 모델	질문·답변과 추적 정보 11개 컬럼	#36 / PR #90
recent_turns	AI에 전달할 최근 성공 대화 조회	#36 / PR #90
create_chat_log	AI 성공·실패 결과 저장과 rollback	#36 / PR #90
list_chat_logs · /api/me/chats	본인 기록 조회와 화면 추가 로딩 지원	#36 / PR #90
테스트·초기화·확인 SQL·ERD	실행 전 준비, 동작 검증, 평가 설명	#36 등

C · 내가 구현
Neon 제공자 선정, development·production DB 브랜치 분리와 pooled 연결 정보의 비공개 전달도 담당했다. Git 브랜치와 Neon DB 브랜치는 서로 다른 것이다. 앱이 어떤 DB를 쓰는지는 연결 정보에 달려 있다.

두 DB 브랜치는 변경을 분리해 개발 데이터가 운영 DB에 직접 쌓이지 않게 할 수 있다. 브랜치를 만드는 것만으로는 부족하고 각 배포 환경에 맞는 연결 정보를 사용해야 한다. [N]

구분해서 기억하기
내 작업과 구분: JWT·비밀번호 해싱·User 모델은 A, AI 호출·messages 구성은 B, HTML·스크롤·공통 스키마·라우터 등록·Vercel 설정은 D다. 나는 이 기능들이 재사용하는 DB 함수와 대화 조회 API를 제공했다.

설명 근거는 작성자 커밋과 현재 코드다. 자료 제작 과정에서 코드를 수정하거나 실제 DB에 접속하지 않았다. 과거 테스트 결과를 오늘 다시 실행한 결과로 표현하지 않는다.

코드·자료 근거: DB 기반 PR #77 / 사용자 CRUD PR #85 / 대화 DB PR #90 / N · Neon 브랜치 설명

12 / 1분 설명과 확인 질문

평가장에서 내 말로 설명하기

평가 답변 예시
로그인 후 보호된 기능을 요청하면 인증 코드가 토큰을 검증하고, 제가 만든 사용자 ID 조회 함수로 DB의 사용자를 확인합니다. 질문을 보내면 제 recent_turns가 그 사용자의 최근 성공 대화를 가져옵니다. 챗봇 담당 코드가 이 기록과 현재 질문으로 AI를 호출하고, 결과는 제 create_chat_log가 DB에 저장합니다. 기록을 볼 때는 로그인한 사용자 ID로만 조회합니다. 저는 DB 연결 기반, 대화 모델, 사용자·대화 CRUD, 기록 API와 검증 도구를 구현했습니다.

확인 질문 · 1. 사용자가 user_id를 조작하면 왜 남의 대화를 볼 수 없나?

기록 API는 쿼리의 user_id를 사용하지 않는다. CurrentUser가 검증한 user.id를 CRUD에 전달하고 WHERE ChatLog.user_id == user_id로 제한한다. 외래키만으로 막는 것이 아니다.

확인 질문 · 2. recent_turns와 list_chat_logs의 차이는?

전자는 AI 문맥용으로 성공 기록만 최근 n개 선택 후 과거순 반환한다. 후자는 화면·조회 API용으로 성공·실패를 모두 최신순 반환하며 limit·offset이 있다. 둘 다 해당 사용자만 조회한다.

확인 질문 · 3. AI 답변을 받았는데 DB 저장이 실패하면?

C 저장 함수가 rollback하고 오류를 전파한다. 정상 ChatResponse를 반환하는 지점에 도달하지 못하므로 공통 500 오류 경로로 간다. 답변을 받았다는 사실만으로 저장 완료라고 할 수 없다.

확인 질문 · 4. 새로고침·로그아웃·재배포 후 기록은?

새로고침은 DB를 재조회한다. 로그아웃은 브라우저 쿠키를 만료시키며 대화 행을 지우지 않는다. 재배포 후에도 같은 외부 DB를 사용하고 데이터가 유지되어 있으면 조회할 수 있다.

확인 질문 · 5. 내 코드를 어디서 보여줄 것인가?

database.py에서 기반, models.py의 ChatLog에서 테이블, crud.py에서 사용자·성공 대화 필터와 저장 순서, routers/logs.py에서 CurrentUser 연결, PostgreSQL 테스트에서 실패 후 세션 재사용을 보여준다.

구분해서 기억하기
자료 기준: 2026-09-17에 확인한 develop b1cca1e. 원격 main 717ca03에도 이 코드가 포함돼 있다. 실제 운영 배포 성공을 이번 자료 제작에서 다시 검증한 것은 아니다. 각 페이지의 코드 근거 링크는 해당 커밋에 고정돼 있다.

코드·자료 근거: 전체 코드 기준 / 내 DB 함수 / 내 조회 API